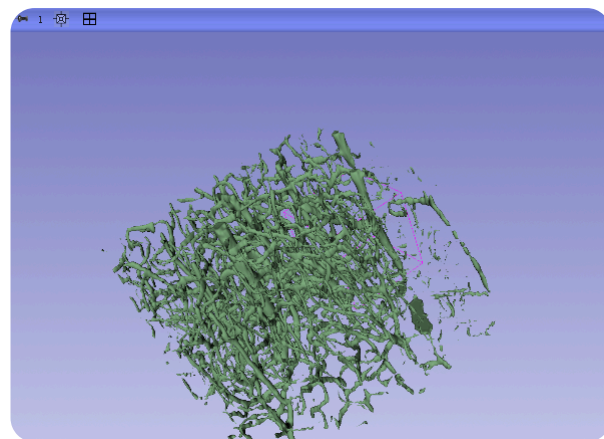




目前我已有的模型差不多都是这样很大一坨的血管，这样应该不方便进行【4.2 STL/OBJ 到 Vascular Centerline Graph 的预处理】吧？我实际的生成需求中，用不着用这么大一坨的血管模型去模拟微泡的运动。从这个角度考虑，对于这类一大坨完整血管的模型，我具体应该怎么处理呢？ 4

 ChatGPT gpt-5-6-thinking

是的，而且从你这张图来看，我会对前面的方案做一个**很重要的缩减**：

这类“完整的一大坨血管”不应该直接作为一个生成样本，也不应该要求模型重新生成同样规模的完整网络。更合理的做法是：把完整血管模型当作“原始血管数据库”，从中自动切出大量适合微泡运动研究的局部血管网络。

这样以后整个课题会清楚很多：

完整血管负责提供真实规律 → 局部血管网络负责训练和生成 → 更小的局部三维模型负责微泡仿真。

这与我们现在的 **Learn → Explain → Generate** 主线并不冲突，反而更符合你的实际应用。

一、先重新定义你手里这种“大血管团”的角色

从你上传的图看，这个模型具有几个很明显的特点：

- 血管数量非常多；
- 分叉密集；
- 大量细血管互相靠得很近；
- 还能看到一些细小的孤立碎片或断裂结构；

- 如果全部作为一张图，节点和 branch 数量可能非常大。

所以如果直接：

text

完整 STL



一次性提取所有精确 centerline



把整张网络作为一个 GNN / Diffusion 样本

工程上会很痛苦，而且与你的最终任务并不匹配。

这并不是说“大型血管图不能做”。已有 whole-brain vessel graph 工作甚至处理到了百万级节点和边，但作者也明确把**超大规模图的处理和拓扑提取**列为主要挑战。(arXiv)

更关键的问题是：

你的微泡并不会同时经过整张脑血管网。

假设一个微泡从某条血管进入，它真正经历的往往只是：

text

入口



一段血管



分叉 1



分叉 2



分叉 3

...

所以你的仿真真正需要的是：

一个具有若干分叉、一定三维曲折度和真实管径变化的局部血管网络。

而不是整个“大毛线团”。

二、因此建议把数据处理改成“两级处理”

以前我们的思路是：

STL → 完整 Vascular Centerline Graph。

现在建议改成：

完整模型只做粗粒度图提取，然后从中切出大量局部子网络，再对局部网络做高质量几何处理。

整个过程变成：



这里最关键的改变是：

“完整血管模型”不是训练样本，而是“训练样本的母体”。

三、为什么不要直接对整坨 STL 做一次高精度 VMTK centerline？

理论上可以尝试，但不建议把它作为第一步的唯一方案。

VMTK 的标准精确 centerline 方法本质上需要确定 source 和 target，然后在血管表面内部计算中心路径；官方教程也是按照入口–出口的方式说明 centerline extraction。([Vascular Modelling Toolkit](#)) 3D Slicer 的 VMTK ExtractCenterline 实现同样需要至少两个端点，并区分 source 和 targets。([GitHub](#))

对于：

text

一条主动脉
+
几个主要分支

这很方便。

但对于你这种：

text

数百 / 数千个终端

如果需要逐个识别出口，就很不现实。

四、所以完整模型第一遍应该只做“粗骨架”

VMTK本身还提供 `vmtknetworkextraction`：

从血管表面直接提取近似 network，作为后续中心线处理的初步网络；其输入表面至少需要存在一个 opening。(VMTK)

也就是说，你可以把处理分成：

第一次

只问：

“这里大概有哪些血管，它们大概怎么连接？”

不追求每个 radius 特别精确。

得到：

text

粗 vascular graph

然后再从里面挑出真正需要的局部区域。

五、在此之前，还应该先处理你图里这种碎片

你这张图上能看到一些细小的孤立绿色碎片。

如果直接 skeletonize：

一个碎片就可能变成一条“假血管”。

于是出现大量：

text

假终点
假分叉
孤立节点

所以第一步应该先做：

Connected Component Cleaning

把 STL 按“互相是否真正连在一起”拆成：

text

Component 1

主血管网络

Component 2


Component 3


Component 4


然后根据：

- surface area；
- volume；
- bounding-box size；
- 后续 centerline length；

识别明显的小碎片。

VMTK的 `vmtksurfaceconnectivity` 本身就支持从表面中提取**最大连通区域**或与指定位置相连的区域。
([VMTK](#))

不过我不建议简单粗暴：

“永远只保留最大的 component。”

因为真实数据可能本来就包含几个不连接的 vascular trees。

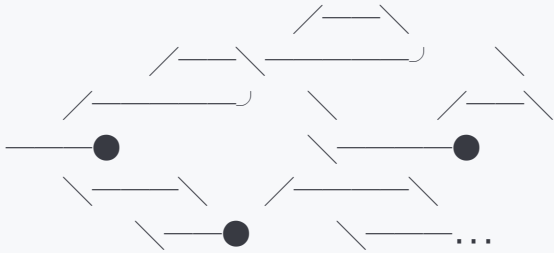
更合理的是：

六、然后才进入最重要的一步：把整张血管网切成“小血管网络”

这是我现在最建议你做的。

例如完整网络：

text

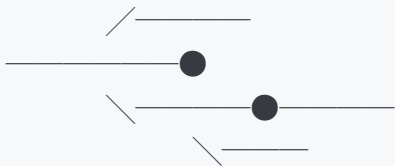


你不要把它作为一个训练样本。

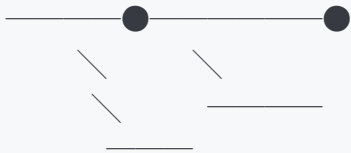
而是自动切成：

text

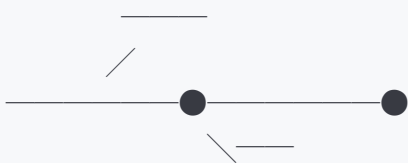
Sample 1



Sample 2



Sample 3



于是：

一个完整脑血管模型，可以变成数百甚至数千个局部训练样本。

这对你目前数据量可能反而是一个优势。

七、关键问题：到底怎么“切”？

有三种方法。

我最推荐第三种。

方法 A：按照空间切

比如在脑血管里放一个：

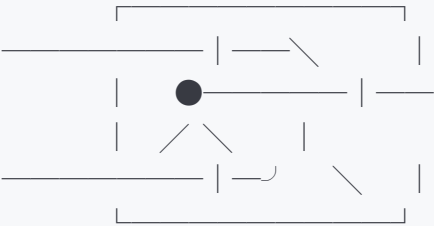
text

$5 \times 5 \times 5 \text{ mm}$

的立方体。

里面所有血管：

text



全部作为一个样本。

优点是简单。

但问题是：

可能把一根血管莫名其妙从中间切断，而且得到的网络没有清晰的“流入 → 流出”方向。

所以不建议作为主要方案。

八、方法 B：以分叉点为中心切

找到一个 bifurcation：

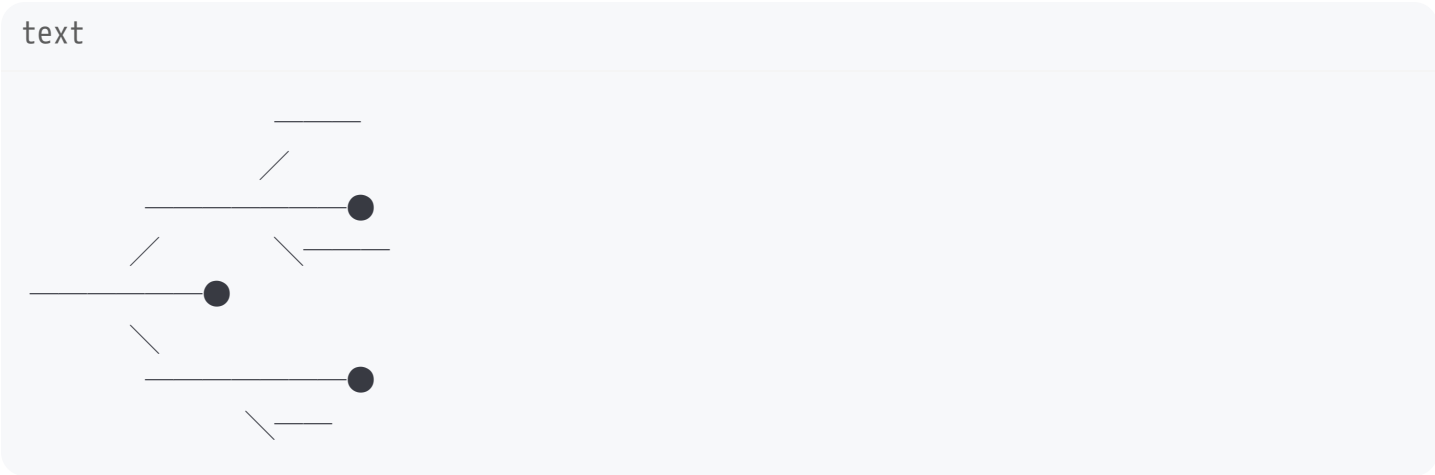
text



然后往：

- 上游取 K 层；
- 下游取 K 层。

例如：



这种特别适合你的：

可解释 GNN。

因为你想研究：

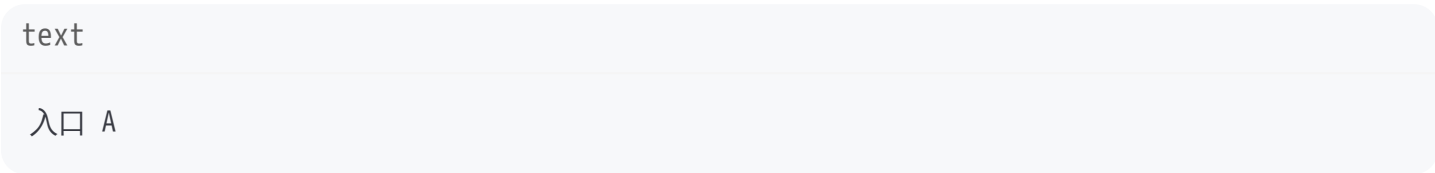
“这个父血管为什么生成这两个孩子？”

那么以分叉为中心的局部图就是天然样本。

九、方法 C：沿“微泡可能经过的血流路径”切

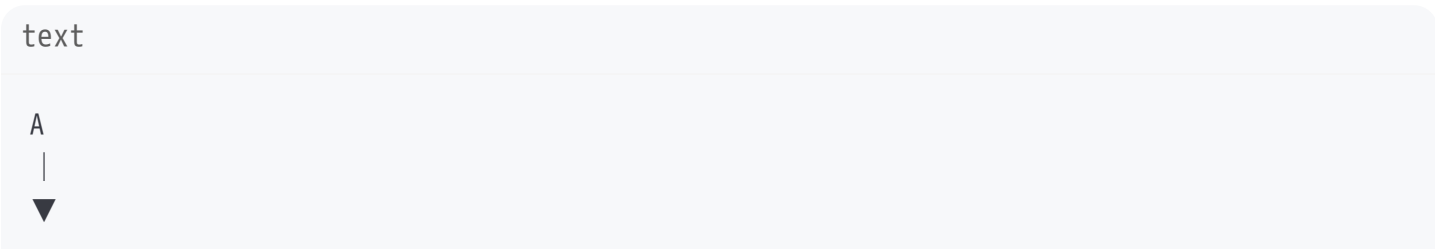
这是我最推荐用于后续生成和微泡仿真的形式。

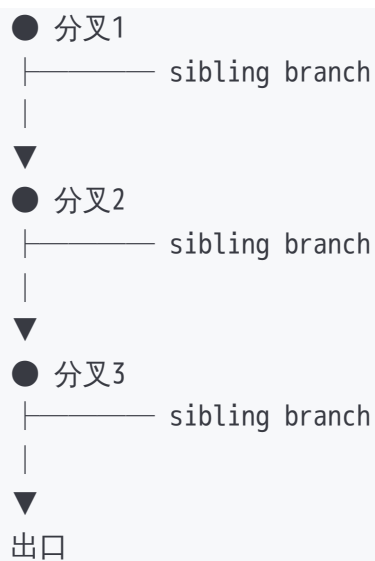
假设微泡从：



进入。

沿着某条血管向前：





那么构造一个：

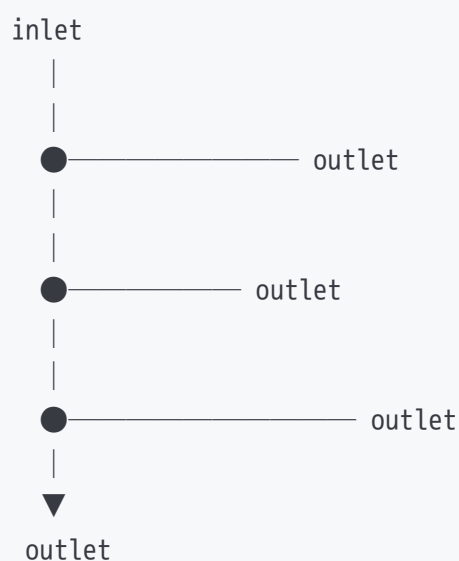
flow-path-centered vascular patch

保留：

- 一条主要路径；
- 路径经过的若干分叉；
- 每个分叉的侧支；
- 必要的上下游缓冲段。

最终得到：

text



这非常适合 CFD / particle tracking。

因为它天然具有：

1 个入口 + 多个出口。

十、这比随便裁一个立方体更适合微泡运动

因为微泡运动真正关心的是：

“它从入口进来以后，在连续几个真实分叉中怎么走？”

所以一个模拟单元可以定义为：

一条主流路径 + K 个连续 bifurcations + 相应侧支。

K 不需要现在固定。

第一阶段可以实验：

```
text

K = 1
K = 3
K = 5
```

看看哪一个复杂度已经足够覆盖你要研究的微泡行为。

这里 K 是“分叉数”，比规定：

10 mm × 10 mm × 10 mm

更有生理/拓扑意义。

十一、这样你的“生成目标”也应该跟着缩小

这是整个项目定位的重要变化。

以前目标是：

生成完整的脑血管网络。

现在我更建议第一篇论文目标变成：

生成具有真实脑血管规律的局部三维血管网络。

英文甚至可以叫：

Local Cerebral Vascular Network Generation

或者：

Generative Modeling of Cerebral Vascular Patches

这个目标对你的微泡模拟更直接。

十二、于是“可解释 GNN”也不需要吃整张脑血管图

这一点很重要。

完整脑血管：

text

几千个 branch

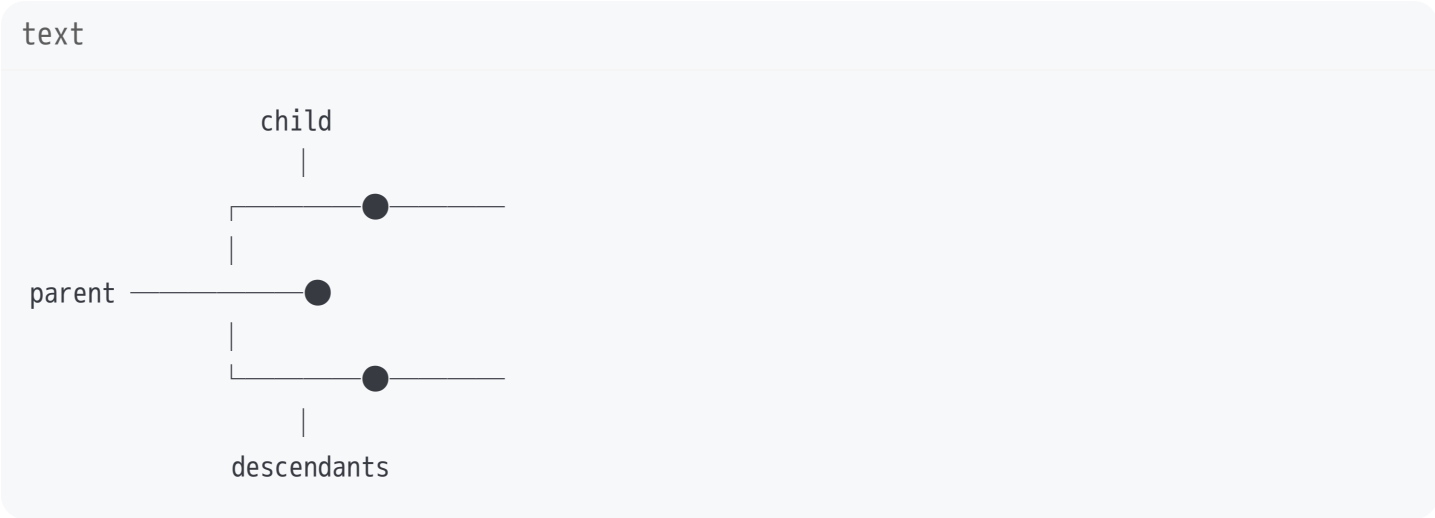
不必一次塞进去。

每个训练样本可以是：

text

中心 bifurcation
+
上游 1-2 层
+
下游 K 层

例如：



GNN依然可以利用：

- parent caliber;
- subtree size;
- branch order;
- 局部 topology;
- 几何信息。

只是从：

global context

变成：

local-to-mesoscale context。

对你的规律发现反而更容易解释。

十三、那 subtree size 这种“全局信息”怎么办？

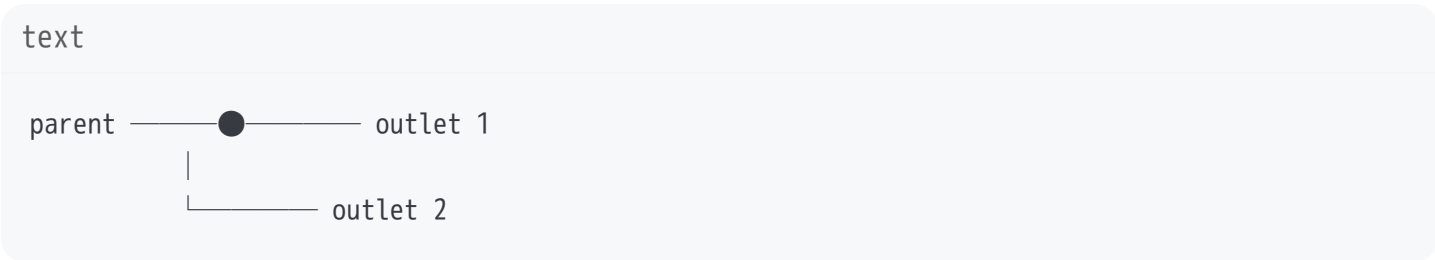
这是个很重要的问题。

即使你只保存局部 patch，也不代表要把完整网络的信息全部丢掉。

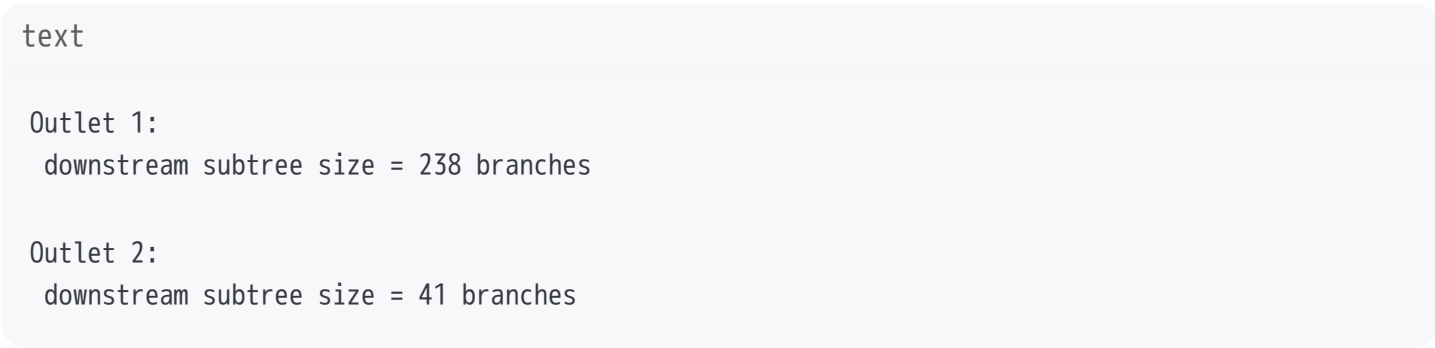
例如原来完整网络：



你裁剪以后：



可以给两个出口保存 metadata：



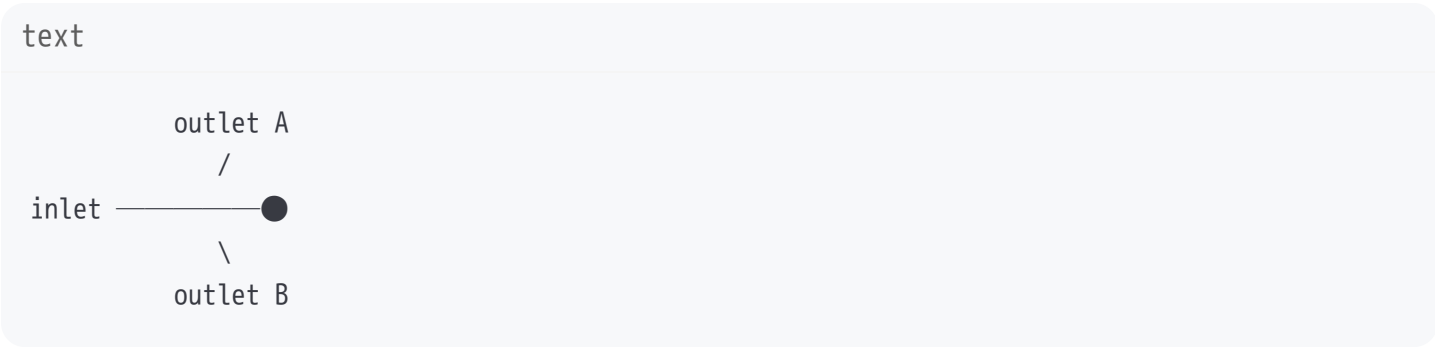
所以：

几何可以裁掉，信息不用全部裁掉。

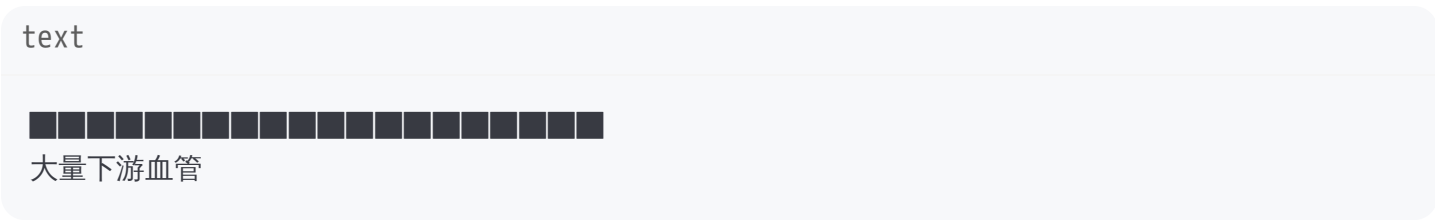
这对可解释 GNN特别有用。

十四、这个思路对后续血流仿真更加重要

当你把完整网络裁成：



你实际上把 outlet A 后面原来的：

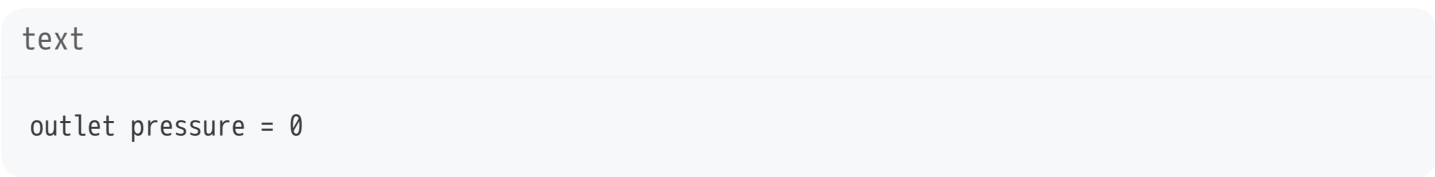


删掉了。

但是这些血管对流动仍然产生：

下游阻力。

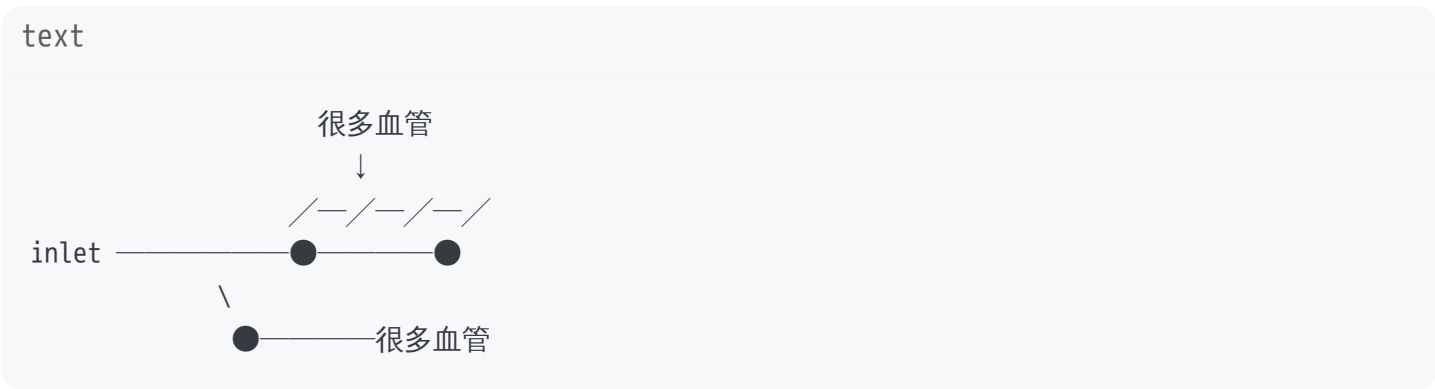
所以不能在 CFD 里简单把：



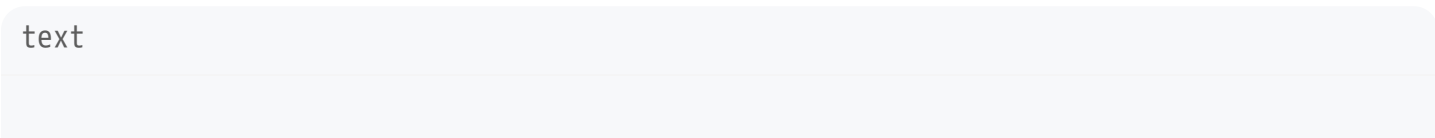
然后认为等价于原网络。

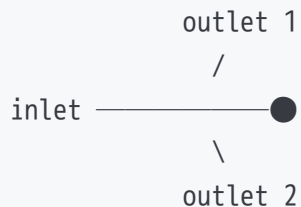
十五、正确做法是：裁掉几何，但保留“等效下游信息”

例如完整网络：



裁成：





同时记录：

```
text

Outlet 1
↓
原来后面有多大的 subtree
↓
Equivalent resistance R1

Outlet 2
↓
Equivalent resistance R2
```

然后局部 CFD / 1D 模型用：

```
text

outlet 1 → R1
outlet 2 → R2
```

模拟被删掉的远端网络。

这本质上就是：

几何局部化，血流边界条件不局部化。

这是以后微泡模拟能不能可信的一个关键点。

SimVascular 的 reduced-order modeling 本身就是根据血管中心线、截面积等信息建立 0D/1D 网络模型，因此局部三维模型与低阶下游网络组合是自然的工作方式。[\(SimVascular\)](#)

十六、我建议最终采用一个“三层模型”

这可能是现在最适合你的架构。

Layer 1：完整血管数据库

```
text

Whole vascular network
```

用途：

提供真实统计规律。

不直接做微泡 CFD。

Layer 2：局部 Vascular Graph

例如：

```
text

1 inlet
3-5 bifurcations
4-8 outlets
```

这里数字只是示例。

用途：

GNN规律发现 + Topology Generator + Geometry Generator。

Layer 3：局部三维 Surface

从 Layer 2：

```
text

centerline
+
radius profile
```

恢复：

```
text

STL
```

用途：

微泡运动 / CFD。

所以：

```
text

Whole model
|
```

▼
知识来源

Local graph



机器学习生成对象

Local STL



微泡仿真对象

这三个层次不要再混在一起。

十七、那么你图里这一大坨血管具体怎么操作？

我建议做以下 **六步**。

Step 1 — Mesh cleanup

对完整 STL：

- connected component 分析；
- 删除明显小碎片；
- 修复 non-manifold；
- 去掉特别短的孤立结构；
- 只做必要平滑。

不要过度 smooth，否则真实的局部形态也会消失。

VMTK提供了基于连通性的表面筛选工具，可用于这一层清理。([VMTK](#))

Step 2 — Voxelization（建议）

对于这种特别密集的表面，我会建议考虑：

text

STL



统一分辨率 voxel mask

然后：

text

voxel skeletonization



粗 vascular graph

原因是：

在超复杂网络中，先求一张全局“粗骨架”往往比尝试为所有末端逐一建立精确 source-target centerline 更现实。

近年的脑血管图分析方法也采用 skeletonization → graph construction 的路线，并能够进一步进行全局或区域级形态统计。(arXiv)

注意：

这张粗 skeleton **不是最终 CFD 几何**。

它只是导航图。

十八、Step 3 – Branch graph

把骨架：

text



压缩成：

text

junction
terminal
branch

也就是：

text



这样完整网络可能从：

text

100,000 个中心线 voxel

降低到：

text

几千条 branch

数据复杂度会下降一个量级。

十九、Step 4 – 自动采样 Local Patches

在整个 branch graph 里：

对规律发现

采样：

text

Bifurcation-centered patch

对微泡生成

采样：

text

Flow-path-centered patch

这样一份大模型可以产生大量数据。

而且必须：

按原始受试者划分 train/test。

不能：

text

同一大脑的 patch A → train

同一大脑的 patch B → test

否则会发生非常严重的数据泄漏。

二十、Step 5 – 回到原 STL 裁精细局部表面

这一点特别重要。

前面 coarse skeleton 只是告诉你：

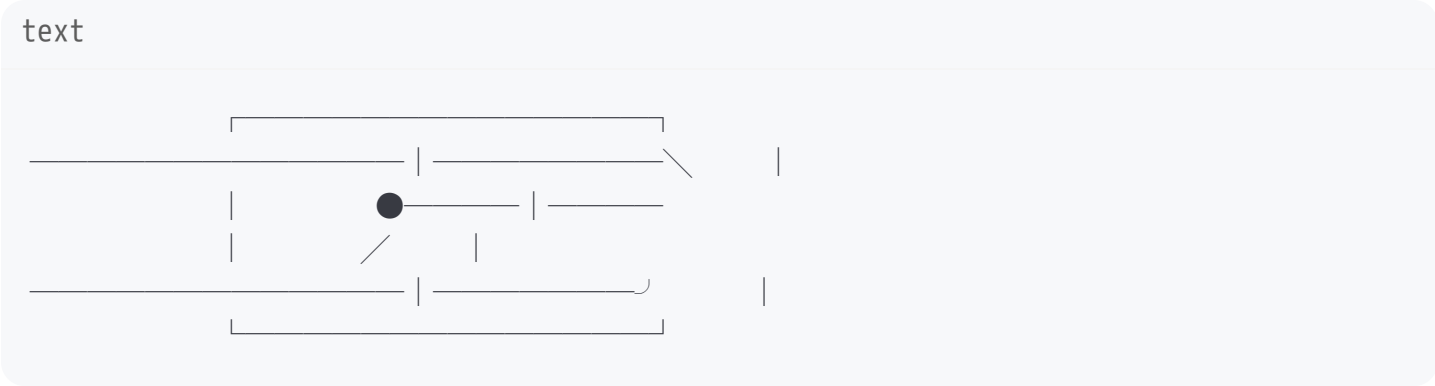
“我要这里。”

例如选中了：

```
text

branch 121
branch 122
branch 123
branch 124
```

根据这些 centerline 建一个稍微扩大的空间包围区：



然后从**原始 STL**中把这一块精确裁出来。

因此不会因为全局 skeleton 粗糙而损失最终几何。

二十一、Step 6 – 局部再做高质量 VMTK centerline

现在模型只有：

```
text

1 inlet
+
几个 outlets
```

这时候 VMTK 就非常适合了。

因为精确 centerline 的 source/target 数量已经可控。VMTK不仅能够计算中心线，还会在中心线/Voronoi 表示中提供 `MaximumInscribedSphereRadius` ，可用于获得沿程局部尺度。[\(Vascular Modelling Toolkit\)](#)

于是最终得到：

```
text

Local Patch
  ↓
高质量 Centerline
  +
```

radius(s)
↓
Local Vascular Centerline Graph

这才是后面真正给机器学习的几何数据。

二十二、这会把我们的整个研究方案再次简化

现在最新版方案可以理解成：

text

数据准备

完整脑血管 STL
↓
粗骨架 / branch graph
↓
自动切出 Local Patches
↓
局部精确 centerline + radius

规律发现

大量 Local Graphs
↓
Interpretable GNN
↓
Symbolic Regression
↓
Vascular Law

生成

Local Topology Generator
↓
新的局部 topology
↓
Law-guided Geometry Generator
↓

centerline + radius profile



Local STL

微泡仿真

Local STL



CFD velocity field



Microbubble tracking

这比“生成整个脑血管再做微泡模拟”合理得多。

二十三、而且它还解决了你一个潜在的数据量问题

假设你现在只有：

text

30 个完整脑血管模型

如果：

一个模型 = 一个训练样本，

那只有 30 个样本。

几乎不适合训练生成模型。

但是如果一个完整模型里有：

text

500 个适合的局部 bifurcation patches

理论上可能得到：

text

$30 \times 500 = 15,000$ 个局部结构

这里的 15,000 只是示例，不是你的实际数据数量。

但核心思想是：

大模型反而是一个“样本矿山”。

当然这些 patch 并不统计独立，所以训练/测试仍必须按**受试者**划分，而不是随机按 patch 划分。

二十四、还有一个更重要的好处：模型的科学问题会更聚焦

以前我们问：

“AI能不能生成一个完整脑血管？”

这个问题太大。

现在问：

“在给定局部拓扑上下文时，真实脑血管通常如何继续分叉和形成几何？”

这个问题：

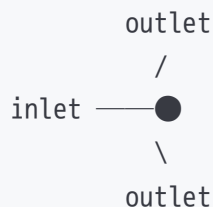
- 可解释 GNN可以回答；
- Symbolic Regression可以回答；
- Geometry Generator可以学习；
- 微泡实验又正好需要。

整个链条终于真正围绕一个空间尺度展开。

二十五、对于你的微泡研究，我建议最后生成的基本单元是这种

而不是一根 Y 型管：

text



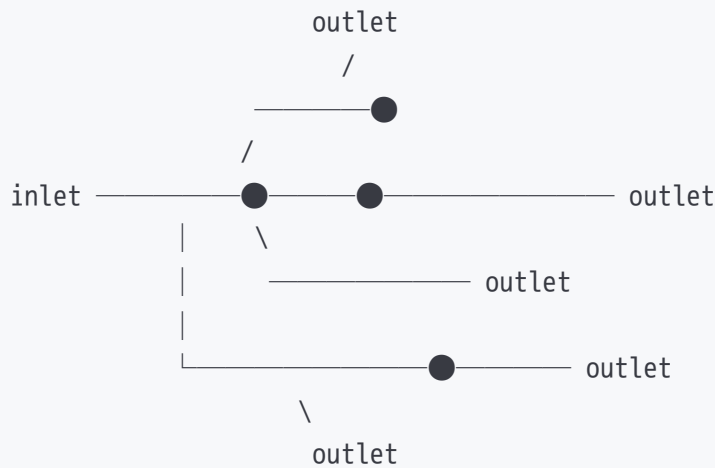
也不要整个脑：

text



而是中间尺度：

text



我会把它叫：

Local Vascular Unit / Local Vascular Patch

它包含：

- 一个明确 inlet；
- 几个连续分叉；
- 多个 outlet；
- 真实 tortuosity；
- 沿程变化的 radius；
- 足够的上下游直段；
- outlet 后面对应的等效下游信息。

这类模型对微泡运动研究的利用率会远高于完整脑网络。

二十六、你还需要根据“微泡到底研究什么”决定 patch 大小

这一点不能由 AI 论文决定，而要由你的物理问题决定。

如果研究：

微泡在单个分叉如何选择 daughter vessel

那么：

1 个主要 bifurcation + 足够上下游长度

可能已经够。

如果研究：

微泡经过连续分叉后的累计路径偏移

那么应该：

多个连续 bifurcations。

如果研究：

微泡在较长网络里的统计输运

那么：

centerline graph + 1D/网络流动

可能比完整 3D CFD 更经济。

如果研究：

微泡靠近血管壁的三维轨迹、局部涡流、声辐射力与局部速度场耦合

那么需要：

局部 3D lumen + CFD。

所以不是：

“越大的血管模型越好。”

而是：

模拟域应该覆盖微泡需要感受到的物理环境，再大就只是增加计算量。

二十七、我会对你的研究目标做一个正式修改

以前：

生成新的三维脑血管模型。

现在建议写成：

从完整真实脑血管模型中学习局部血管网络的拓扑与几何规律，并生成具有真实人群统计特征的三维局部脑血管网络，为微泡输运与血流动力学模拟提供可控的虚拟血管样本。

这会比：

“生成整个脑血管”

和你的最终应用联系得更紧密。

二十八、现在最值得做的第一个原型实验

暂时不要训练任何 GNN，也不要训练 Diffusion。

直接拿你图里这一份代表性 STL 做：

1. **统计 connected components**，看看有多少明显碎片。
2. 将清理后的主网络**体素化并提取粗 skeleton**。
3. skeleton 转成 branch graph。
4. 随机选 20 个 bifurcations。
5. 每个 bifurcation 提取一个 2-hop / 3-hop 局部图。
6. 回到原 STL 把这 20 个区域裁出来。
7. 在局部 STL 上重新提取精确 centerline 和 radius profile。
8. 检查这些局部模型是否已经是愿意拿去做微泡仿真的几何尺度。

如果第 8 步答案是“是”，那么你的整个项目的数据单位就确定了：

不是 Whole Brain Vessel，而是 Local Vascular Patch。

这一步会直接决定后面的 **可解释 GNN输入是什么、Topology Generator生成什么、Geometry Generator输出什么，以及最终 CFD 到底模拟什么。**

而基于你现在展示的完整血管模型和“无需整脑进行微泡运动仿真”这一实际需求，我认为把**完整网络作为规律来源，把局部网络作为生成和仿真对象**，是目前这套研究方案最值得做的一次调整。